

A SOURCE CODE FOR BENCHMARKED PROGRAMS

```

1 1 let filter =
2 2   let edgeDetect = [[0, 1, 0],
3 3                     [1, -4, 1],
4 4                     [0, 1, 0]] in
5 5   [[ nth2 i j edgeDetect | (i, j) in (3, 3) ]];
6 6   convolve input_image filter extend
7
8 (a) edgeDetect

```

```

9 1 let filter =
10 2   let emboss = [[-2, -1, 0],
11 3                 [-1, 1, 1],
12 4                 [0, 1, 2]] in
13 5   [[ nth2 i j emboss | (i, j) in (3, 3) ]];
14 6   convolve input_image filter zero
15
16 (b) emboss

```

```

17 1 let filter =
18 2   let gaussian = [[1, 4, 1],
19 3                   [4, 16, 4],
20 4                   [1, 4, 1]] in
21 5   [[ nth2 i j gaussian | (i, j) in (3, 3) ]];
22 6   convolve input_image filter zero
23
24 (c) gaussian

```

```

25 1 let zero n = const n;
26 2   extend n = min (max n 1);
27 3 let convolve image kernel method =
28 4   let ((m, n), (i, j)) = (dims image, dims kernel);
29 5   (half_i, half_j) = (i `quot` 2, j `quot` 2);
30 6   area = i * j
31 7   in [[ let weightedSum = sum [
32 8         image!(x, y) * kernel!(i' + 1, j' + 1)
33 9         | (i', j') ← range (0, 0) (i - 1, j - 1),
34 10        let x = method (m' + i' - half_i) m,
35 11        let y = method (n' + j' - half_j) n,
36 12        x >= 1, x <= m, y >= 1, y <= n
37 13        ] in weightedSum `quot` area
38 14        | (m', n') in (m, n) ]];
39 15 let input_image =
40 16   let image = [[15, 13, 6, 9, 16],
41 17                [12, 5, 15, 4, 13],
42 18                [14, 9, 20, 8, 1],
43 19                [4, 10, 3, 7, 19],
44 20                [3, 11, 15, 2, 9]] in
45 21   [[ nth2 i j image | (i, j) in (5, 5) ]];
46
47 (d) Library code

```

Fig. 1. Source code: convolution examples

```

50 1 MultiPlot {
51 2   "bar-chart" :=
52 3     let totalFor c rows =
53 4       sum [ row.output | row ← rows, row.country == c ];
54 5     let data2015 = [ row | row ← renewables, row.year == 2015 ];
55 6     countryData = [ { x: c, bars: [ { y: "output", z: totalFor c data2015 } ] }
56 7                   | c ← ["China", "USA", "Germany"] ]
57 8     in BarChart {
58 9       caption: "Total output by country",
59 10      data: countryData
60 11    },
61 12   "line-chart" :=
62 13     let series type country = [
63 14       { x: row.year, y: row.output }
64 15       | year ← [2013..2018], row ← renewables,
65 16       row.year == year, row.energyType == type, row.country == country
66 17     ] in LineChart {
67 18       caption: "Output of USA relative to China",
68 19       plots: [
69 20         LinePlot { name: type, data: plot }
70 21         | type ← ["Bio", "Hydro", "Solar", "Wind"],
71 22         let plot = zipWith (fun p1 p2 → { x: p1.x, y: p1.y / p2.y })
72 23           (series type "USA") (series type "China")
73 24       ]
74 25     }
75 26 }

```

Fig. 2. Source code: bar-chart-line-chart

```

99 1  MultiPlot {
100 2    "stacked-bar-chart" :=
101 3      let totalFor year country rows =
102 4        let [ row ] = [ row | row ← rows, row.year == year, row.country == country ]
103 5        in row.nuclearOut + row.gasOut + row.coalOut + row.petrolOut;
104 6      let stack year = [ { y: country, z: totalFor year country nonRenewables }
105 7        | country ← ["BRA", "EGY", "IND", "JPN"] ]
106 8      in BarChart {
107 9        caption: "Non-renewables by country",
108 10       data: [ { x: numToStr year, bars: stack year }
109 11         | year ← [2014..2018] ]
110 12     },
111 13   "scatter-plot" :=
112 14     let isCountry name x = name == x.country;
113 15     let isYear year x = year == x.year;
114 16
115 17     let plot year countries =
116 18       let rens = filter (isYear year) renewables;
117 19       nonRens = filter (isYear year) nonRenewables;
118 20       let plotCountry country =
119 21         let rens' = filter (isCountry country) rens;
120 22         rensOut = sum (map (fun x → x.output) rens');
121 23         rensCap = sum (map (fun x → x.capacity) rens');
122 24         x = head (filter (isCountry country) nonRens);
123 25         nonRensCap = x.nuclearCap + x.petrolCap + x.gasCap + x.coalCap
124 26         in {
125 27           x: rensCap / (rensCap + nonRensCap),
126 28           y: (rensOut + x.nuclearOut) / (rensCap + x.nuclearCap)
127 29         }
128 30     in map plotCountry countries
129 31
130 32   in ScatterPlot {
131 33     caption: "Clean energy efficiency vs proportion of renewable energy capacity",
132 34     data: plot 2018 [ "BRA", "CHN", "DEU", "FRA", "EGY"
133 35       , "IND", "JPN", "MEX", "NGA", "USA" ],
134 36     xlabel: "Renewables/TotalEnergyCap",
135 37     ylabel: "Clean Capacity Factor"
136 38   }
137 39 }

```

Fig. 2. Source code: stacked-bar-scatter-plot

```

148 1 let year = 2015;
149 2 let exclude countries yearData =
150 3   flip map yearData
151 4     (second (filter (fun (country, countryData) → not (elem country countries))))
152 5 in caption ("Renewables (GW) by country and energy type, " ++ numToStr year)
153 6   (groupedBarChart True colours1 0.2
154 7     (exclude [] (lookup year data)))

```

(a) grouped-bar-chart

```

156 1 let year1 = head data;
157 2 let addTotal kvs =
158 3   ("Total", sum (map snd kvs)) : kvs;
159 4 let countryData country =
160 5   map (second (compose addTotal (lookup country))) data in
161 6 (fun country →
162 7   caption country
163 8     (lineChart True ("black" : colours1) (fst year1)
164 9       (countryData country)))
165 10 "China"

```

(b) line-chart

```

167 1 let year = 2015 in
168 2 caption ("Total renewables (GW) by country, " ++ numToStr year)
169 3   (stackedBarChart True colours1 0.2 (lookup year data))

```

(c) stacked-bar-chart

Fig. 3. Source code: graphics examples

```

197 1  let coords (Group gs) = Point 0 0;
198 2      coords (Rect x y _ _ _) = Point x y;
199 3      coords (Text x y _ _ _) = Point x y;
200 4      coords (Viewport x y _ _ _ _ _ _) = Point x y;
201 5  let get_x g = let Point x _ = coords g in x;
202 6  let get_y g = let Point _ y = coords g in y;
203 7  let set_x x (Group gs) = error "Group has immutable coordinates";
204 8      set_x x (Rect _ y w h fill) = Rect x y w h fill;
205 9      set_x x (Text _ y str anchor baseline) = Text x y str anchor baseline;
206 10      set_x x (Viewport _ y w h fill margin scale translate g) =
207 11          Viewport x y w h fill margin scale translate g;
208 12 let dimensions2 (Point x1 y1, Point x2 y2) = Point (max x1 x2) (max y1 y2);
209 13 let dimensions (Group gs) = foldl (curry dimensions2) (Point 0 0) (map (coords_op) gs);
210 14     dimensions (Polyline ps _ _) = foldl (curry dimensions2) (Point 0 0) ps;
211 15     dimensions (Rect _ _ w h _) = Point w h;
212 16     dimensions (Text _ _ _ _ _) = Point 0 0;
213 17     dimensions (Viewport _ _ w h _ _ _ _) = Point w h;
214 18 let coords_op g =
215 19     let (Point x y, Point w h) = prod coords dimensions g in
216 20     Point (x + w) (y + h);
217 21 let width g = let Point w _ = dimensions g in w;
218 22 let height g = let Point _ h = dimensions g in h;
219 23 let spaceRight z sep gs =
220 24     zipWith set_x (iterate (length gs) ((+) sep) z) gs;
221 25 let colours1 = [ "#66c2a5", "#a6d854", "#ffd92f", "#e5c494"
222 26                 , "#fc8d62", "#b3b3b3", "#8da0cb", "#e78ac3"];
223 27 let colours2 = [ "#e41a1c", "#377eb8", "#4daf4a", "#984ea3"
224 28                 , "#ff7f00", "#ffff33", "#a65628", "#f781bf"];
225 29 let scaleToWidth w (Rect x y _ h fill) = Rect x y w h fill;
226 30     scaleToWidth w (Viewport x y w0 h fill margin (Scale x_scale y_scale) translate g) =
227 31         let scale = Scale (x_scale * w / w0) y_scale in
228 32         Viewport x y w h fill margin scale translate g;
229 33 let stackRight sep gs =
230 34     map (scaleToWidth (1 - sep)) (spaceRight (sep / 2) 1 gs);
231 35 let groupRight sep gs =
232 36     Viewport 0 0 (length gs) (maximum (map height gs)) "none" 0 (Scale 1 1)
233 37     (Translate 0 0) (Group (stackRight sep gs));
234 38 let tickEvery n =
235 39     let m = floor (logBase 10 n) in
236 40     if n <= 2 * 10 ** m then 2 * 10 ** (m - 1) else 10 ** m;

```

Fig. 3. Source code: library code for graphics examples (1 of 3)

```

246 1  let axisStrokeWidth = 0.5;
247 2    axisColour = "black";
248 3    backgroundColour = "white";
249 4    defaultMargin = 24;
250 5    markerRadius = 3.5;
251 6    tickLength = 4;
252 7  let tick Horiz colour len = Line (Point 0 0) (Point 0 (0 - len)) colour axisStrokeWidth;
253 8    tick Vert colour len = Line (Point 0 0) (Point (0 - len) 0) colour axisStrokeWidth;
254 9  let label Horiz x distance str = Text x (0 - distance - 4) str "middle" "hanging";
255 10    label Vert x distance str = Text (0 - distance) x str "end" "central";
256 11  let labelledTick orient colour len str =
257 12    Group [tick orient colour len, label orient 0 len str];
258 13  let mkPoint Horiz x y = Point y x;
259 14    mkPoint Vert x y = Point x y;
260 15  let axis orient x start end =
261 16    let tickSp = tickEvery (end - start);
262 17      firstTick = ceilingToNearest start tickSp;
263 18      lastTick = ceilingToNearest end tickSp;
264 19      n = floor ((end - firstTick) / tickSp) + 1;
265 20      ys = iterate n ((+) tickSp) firstTick;
266 21      ys = if firstTick > start then start : ys else ys;
267 22      ys = if lastTick > end then concat2 ys [lastTick] else ys;
268 23      ps = map (mkPoint orient x) ys;
269 24      ax = Group [
270 25        Line (head ps) (last ps) axisColour axisStrokeWidth,
271 26        Polymarkers ps (flip map ys (compose (labelledTick orient axisColour tickLength)
272 27          numToStr))
273 28      ]
274 29    in (ax, lastTick);
275 30  let catAxis orient x catValues =
276 31    let ys = iterate (length catValues + 1) ((+) 1) 0;
277 32      ps = map (mkPoint orient x) ys
278 33    in Group [
279 34      Line (head ps) (last ps) axisColour axisStrokeWidth,
280 35      Polymarkers (tail ps) (map (const (tick orient axisColour tickLength)) catValues),
281 36      Polymarkers (flip map (tail ps) (fun (Point x y) → Point (x - 0.5) y))
282 37        (map (label orient -0.5 0) catValues)
283 38    ];

```

Fig. 3. Source code: library code for graphics examples (2 of 3)

```

295 1 let viewport x_start x_finish y_finish margin gs =
296 2   Viewport 0 0 (x_finish - x_start) y_finish backgroundColour margin
297 3   (Scale 1 1) (Translate (0 - x_start) 0) (Group gs);
298 4 let lineChart withAxes colours x_start data =
299 5   let xs = map fst data;
300 6   nCat = length (snd (head data));
301 7   let plot (n, colour) =
302 8     let ps = map (fun (x, kvs) → Point x (snd (nth n kvs))) data
303 9     in Group [
304 10      Polyline ps colour 1,
305 11      Polymarkers ps (repeat (length ps) (Circle 0 0 markerRadius colour))
306 12    ];
307 13 let lines = zipWith (curry plot) (iterate nCat ((+ 1) 0) colours;
308 14   x_finish = last xs;
309 15   y_finish = maximum (flip map data (fun (_, kvs) → maximum (map snd kvs)))
310 16 in if withAxes
311 17 then
312 18   let (x_axis, x_finish) = axis Horiz 0 x_start x_finish;
313 19   (y_axis, y_finish') = axis Vert x_start 0 y_finish
314 20   in viewport x_start x_finish y_finish' defaultMargin (x_axis : y_axis : lines)
315 21 else viewport x_start x_finish y_finish 0 lines;
316 22 let categoricalChart plotValue withAxes colours sep data =
317 23   let gs = stackRight sep (plotValue colours (map snd data));
318 24   w = length gs;
319 25   h = maximum (map height gs)
320 26 in if withAxes
321 27 then
322 28   let x_axis = catAxis Horiz 0 (map fst data);
323 29   (y_axis, h') = axis Vert 0 0 h
324 30   in viewport 0 w h' defaultMargin (concat2 gs [x_axis, y_axis])
325 31 else viewport 0 w h 0 gs;
326 32 let rects colours ns = zipWith (fun colour n → Rect 0 0 1 n colour) colours ns;
327 33 let stackedBar colours ns =
328 34   let heights = map snd ns;
329 35   subtotals = scanl1 (+) 0 heights;
330 36   dims = zip (0 : subtotals) heights;
331 37   rects = map
332 38     (fun ((y, height), colour) → Rect 0 y 1 height colour)
333 39     (zip dims colours)
334 40   in Viewport 0 0 1 (last subtotals) "none" 0 (Scale 1 1) (Translate 0 0) (Group rects);
335 41 let barChart = categoricalChart rects;
336 42 let groupedBarChart = categoricalChart (compose map (flip (barChart False) 0));
337 43 let stackedBarChart = categoricalChart (compose map stackedBar);
338 44 let caption str (Viewport x y w h fill margin scale translate g) =
339 45   let g' = Group [ Text (x + w / 2) -2 str "middle" "hanging",
340 46     Viewport 0 0 w h fill margin scale translate g ]
341 47 in Viewport x y w h backgroundColour (defaultMargin / 2 + 4) (Scale 1 1) (Translate 0 0) g';
342
343

```

Fig. 3. Source code: library code for graphics examples (3 of 3)